

Review

Low-Resource Thai Profanity Detection of Speech Recognition

Nutchapol Rodpholchoo^{1,a}, Nuchanon Tongkumbanchong^{1,b}, and Sirinalin Saekhow^{1,c}

¹ Department of Computer Science and Information Technology, Vincent Mary School of Engineering, Science and Technology, Assumption University, Thailand

E-mail: ^a[nutchapol.r@gmail.com], ^b[u6511074@au.edu], ^c[u6510240@au.edu]

Abstract. This study develops an initial Thai profanity detection model and, more importantly, identifies effective evaluation settings for practical censorship. We fine-tune a Thai wav2vec 2.0 classifier and perform a systematic parameter study over window sizes (0.3–2.0 s) and two stride schemes—fixed 0.05 s and percentage-of-window—assessed under four protocols: (i) window-level classification, (ii) merged-word evaluation, (iii) merged-word with Intersection-over-Union (IoU) thresholds, and (iv) variants incorporating voice activity detection (VAD) for boundary refinement. Across a large grid of configurations, three findings emerge: shorter windows (~0.3 s) improve word localization; percentage-based strides maintain consistent overlap across window sizes; and merging with an IoU threshold stabilizes decisions relative to raw window predictions. VAD further trims silence and bridges brief gaps, yielding cleaner, censorable spans. The outcome is an initial, deployable model together with concrete, evidence-based parameter recommendations and a reusable evaluation framework. These results reduce ad hoc tuning, guide future work on Thai profanity detection under data scarcity, and accelerate real-world deployment of automated beep-censoring systems.

Keywords: Hate Speech, Profanity Detection, Thai, Wav2Vec2, Voice Activity Detection, Speech Recognition, Keyword spotting

ENGINEERING JOURNAL Volume # Issue #

Received Date Month Year

Accepted Date Month Year

Published Date Month Year

Online at <http://www.engj.org/>

DOI:10.4186/ej.20xx.xx.x.xx

1. Introduction

Content moderation remains a critical challenge for digital media platforms that host large volumes of explicit content, especially in languages with limited computational resources such as Thai. While automated profanity detection systems are well-researched for languages like English in both text [1] and audio [2], Thai presents unique computational challenges due to its tonal nature and complex phonetic structure [3], as well as the scarcity of large-scale profanity-labeled datasets. Moreover, traditional manual censorship is still widely used, but it is time-consuming, especially for large-scale content moderation.

Recent advances in speech recognition and deep learning have enabled more sophisticated content moderation tools. However, most existing models are trained on non-Thai datasets. Therefore, our work addressed these challenges by fine-tuning a pre-trained wav2vec 2.0 trans model [4] specifically for Thai profanity detection through supervised learning with sequence classification. The base transformer model used was `airesearch/wav2vec2-large-xlsr-53-th`, which had been pre-trained on Thai speech data using self-supervised learning techniques.

While this initial system demonstrated promising results, its performance was constrained by limited training data, time, and computational resources. A fundamental issue in low-resource language processing is that deep learning models typically require extensive labeled datasets to achieve optimal performance.

The development of robust Thai profanity detection models faces this central constraint: the lack of extensive labeled datasets. Although ASR-based pipelines show promise, they struggle with Thai's phonetic diversity, slang variations, and tonal ambiguities, leading to inconsistent profanity detection accuracy [3, 5]. More critically, the evaluation methodologies used to assess these systems often lack systematic analysis of temporal parameters, specifically window size and stride that significantly impact detection performance in practical deployment scenarios [2, 6, 7].

Traditional evaluation approaches in speech processing often use fixed parameters without exploring the relationship between temporal segmentation settings and detection accuracy in detail. In profanity detection, the choice of analysis window size and stride directly affects both detection sensitivity and temporal precision, yet existing evaluation methods rarely provide guidance on appropriate stride for parameter optimization [4], especially for Thai where phoneme and tone differences are critical [3, 5].

Recognizing the data limitation as an inherent challenge rather than a solvable problem within the scope of our research, this project shifts focus from purely model development to comprehensive evaluation methodology and parameter optimization. The central question becomes: given

limited training data, how can we systematically identify the most effective evaluation approaches and optimal system parameters (window size and stride configurations) to guide future research and practical implementation?

This work conducts a systematic evaluation of temporal segmentation choices for Thai profanity detection in audio. We design and compare multiple evaluation protocols and analyze the effect of window size (0.3–2.0 s) and stride schemes (fixed 0.05 s and percentage-of-window) on detection and temporal localization performance.

Our contributions are: (i) an empirical study quantifying how window size and stride influence F1 and IoU under window-based and IoU-based evaluations; (ii) scenario-oriented parameter recommendations for censorship use (e.g., short windows with high overlap for precise timestamps, larger windows for presence detection); and (iii) a reproducible evaluation toolkit that aligns preprocessing with training to facilitate consistent comparison in future work.

2. Related Work

Research on spoken profanity/hate-speech detection spans two main paradigms: (i) cascaded ASR pipelines (including speech-native alternatives like acoustic word embeddings, AWE) and (ii) direct audio methods. Text-only approaches [8, 9, 1] are relevant for moderation but do not yield timestamp-accurate spans required for censoring.

ASR vs. AWE. Jacobs et al. [10] compare cascaded ASR systems with AWE models for low-resource languages (Wolof, Swahili), showing that AWE can match or surpass ASR in the presence of transcription errors by operating directly on audio and avoiding error propagation. This supports speech-native representations when labeled text resources are scarce, and motivates evaluation settings that emphasize temporal localization from audio.

Direct audio methods. CNN/RNN classifiers on spectrograms have been used for profanity detection [2] and broadly for audio event recognition [11], such as wav2vec2.0 operate on raw waveforms E2E CNN and provide fine temporal sensitivity [4]. With self-supervised model cross-lingual extensions improving transfer across languages [12, 13], enabling prove of utilizing pre-trained transformer model.

Keyword spotting (KWS). KWS targets predefined terms with low-latency, small-footprint models [6], commonly using CNNs over log-mel features [14] and widely adopted datasets such as Speech Commands [15]. KWS-style post-processing (merging adjacent positives) is useful for forming robust word-level spans.

Temporal segmentation and evaluation. Prior work has examined fixed window lengths (0.3–0.5 s) and overlaps for censorship [2], but systematic analyses of stride relative to window size are limited. Localization-quality metrics from sound event detection (e.g., event/segment measures

and IoU) provide objective temporal scoring [7, 16]. These gaps motivate our study of window/stride trade-offs under window- and IoU-based evaluations tailored to censorable spans.

3. Methodology

3.1. Data Collection

We collected data by recording speech and downloading public audio/video content, then labeled profane words in Audacity with start/end timestamps and labels. Labels were consolidated in a CSV containing file path, start time, end time, and label.

Table 1. Example of label storage format used in the CSV.

file_path	start_time	end_time	label
./dataset/audio/example.wav	0	14.18	none
./dataset/audio/example.wav	14.18	14.91	profanity1
./dataset/audio/example.wav	14.91	15.37	profanity2

Table 2. Dataset summary: counts and total duration per label.

Label	Count	Total Duration (s)
none	1969	3088.47
guu	290	51.58
meung	254	52.61
hia	164	48.33
yed	76	15.45

Table 3. Dataset totals used for training.

Profanity instances (sum of Thai labels)	784
Non-profanity instances	1969
Total instances	2753

Table 4. Evaluation dataset summary: counts and total duration per label.

Label	Count	Total Duration (s)
none	298	3369.74
guu	157	19.14
meung	117	17.36
hia	78	16.52
yed	26	2.43

Table 5. Evaluation dataset totals.

Profanity instances (sum of profanity labels)	378
Non-profanity instances	298
Total instances	676
Number of unique source files	34

3.2. Data Processing

We match the deployed and evaluation pipelines to the training-time preprocessing used for fine-tuning `airesearch/wav2vec2-large-xlsr-53-th`. For each windowed slice, we load the exact segment via frame offsets to avoid drift, apply a first-order pre-emphasis filter ($\text{coef} = 0.97$), and perform a gentle energy-based gate by attenuating samples below $2.5\times$ the 20th-percentile amplitude. To reduce boundary artifacts without suppressing content, we use a Hamming window technique [17, 18, 19], followed by mild dynamic range compression via soft clipping around $2\times$ RMS to stabilize dynamics across windows. Audio is then resampled to 16 kHz mono if needed, normalized by z-scoring (mean/std) and scaled by 0.5 to match training amplitude statistics, and finally padded or truncated to the exact window length in samples (e.g., $0.5\text{ s} \Rightarrow 8000$ samples at 16 kHz). For training, we segment windows directly from labeled timestamps. For evaluation and real usage, we slide windows with overlap to avoid splitting words [2], merge adjacent positives into word-level spans, and optionally refine boundaries with a simple VAD via `librosa` utilities [20] or a statistical detector [21] (we use `librosa.effects.split, top_db=20`).

3.3. Model

We adopt a Thai `wav2vec 2.0` backbone (`airesearch/wav2vec2-large-xlsr-53-th`) [4] and fine-tune it for five-way sequence classification with labels `{none, yed, guu, meung, hia}`. At inference time, we run the classifier on overlapping fixed-length windows and then merge contiguous positive predictions into word-level spans (see Fig. 2). Below we describe each component and why it is used, without equations.

3.4. Training

Our training methodology leverages transfer learning to address the low-resource nature of Thai profanity detection. We fine-tune the Thai `wav2vec 2.0` backbone, which provides strong speech representations learned from general Thai audio, and adapt it to multi-class profanity classification. Training is configured to balance learning efficiency and generalization: a batch size of 16 offers stable gradients within available compute; a learning rate of $5e-5$ with linear decay enables fine-grained updates without overshooting; and AdamW with decoupled weight decay regularizes

parameters to reduce overfitting [22]. To handle class imbalance, we use class-weighted cross-entropy with weights inversely proportional to class frequency, so rare profanity classes contribute more to the loss. We train for up to 100 epochs with early stopping based on validation performance.

Data augmentation. We adopt a targeted but comprehensive augmentation suite to expose the model to realistic acoustic variability likely to occur in production: (i) time stretching within a $\pm 10\%$ range to accommodate speaking rate differences; (ii) pitch shifting by ± 2 semitones to simulate speaker characteristics and vocal ranges; and (iii) random time and frequency masking to discourage over-reliance on specific temporal cues and promote invariance to local dropouts (SpecAugment-style masking) [24, 23].

Data selection. To reflect real-world usage in which profanity must be detected and censored in online media, we construct our splits from Recording, YouTube and other public sources. Audio is collected from diverse channels and content genres (dialogue, livestreams, vlogs, gaming, and music-backed speech) and segmented into clips with annotated profane spans and non-profane speech. For training contain voice recordings and YouTube content, while validation and test sets are held out from distinct YouTube channels. This split strategy evaluates generalization to unseen speakers and contexts, reflecting practical deployment scenarios.

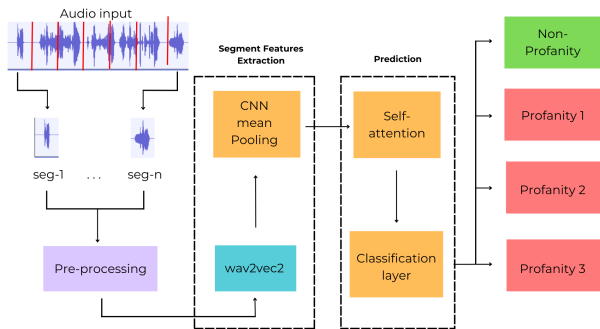


Fig. 1. Model framework overview

Model framework overview. The profanity detection system processes audio input through several stages to produce segment-level predictions (see Fig. 1):

- **Audio Segmentation and Pre-processing:** The input audio is divided into overlapping segments (windows) of fixed duration. Each segment undergoes pre-processing, including normalization and optional noise reduction, to standardize input conditions and enhance feature extraction.
- **Feature Extraction (wav2vec2):** Each pre-processed segment is passed through a wav2vec2

feature encoder. This module consists of a convolutional neural network (CNN) that extracts low-level acoustic features, followed by a stack of transformer layers that model long-range temporal dependencies via self-attention.

- **Pooling and Classification:** The segment-level representations are aggregated using mean pooling. The pooled features are then fed into a classification head, which consists of a fully connected layer (or layers) that outputs class probabilities for each segment.
- **Prediction:** The model predicts one of several classes for each segment: non-profanity or one of multiple profanity types. The final output is a sequence of segment-level predictions, which can be post-processed (e.g., merged, filtered) to yield word-level or region-level profanity spans.

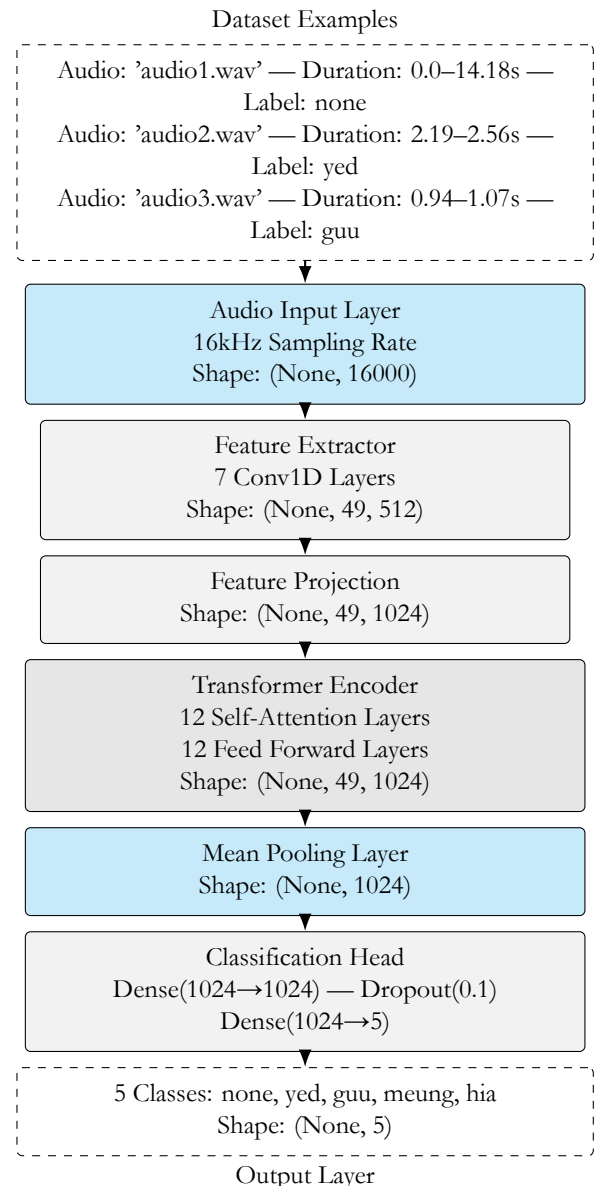


Fig. 2. Model framework overview (5-label setup)

From Figs. 2, the model architecture consists of the following components: **Input and windowing.** Audio is resampled to 16 kHz mono and segmented into overlapping windows (for example, 0.3–2.0 seconds) using the pre-processing pipeline described earlier. Windowing provides temporal granularity for both detection and timestamping and lets us trade off latency and boundary precision.

CNN feature encoder (7 conv1d layers). The raw waveform first passes through a stack of seven 1D convolution layers with progressively larger stride. This converts samples into a compact sequence of latent frames at a lower frame rate (roughly a few dozen frames per second). Convolutions are local and efficient; they learn robust low-level acoustics (onsets, formants, energy patterns) while down-sampling to reduce the length of the sequence the transformer must process. This stage corresponds to the “Feature Extractor” in the diagram.

Feature projection and positional embedding. A linear projection lifts the convolutional features to the transformer width (1024 channels), followed by normalization and dropout. A learned convolutional positional embedding is added so the transformer is aware of frame order. This step aligns the representation with the transformer’s expected dimensionality and improves stability.

Transformer encoder (12 layers). We use a stack of twelve transformer blocks with multi-head self-attention and feed-forward sublayers, each wrapped with residual connections and layer normalization. Self-attention lets the model integrate information across the entire window, capturing long-range dependencies (for example, phoneme context and coarticulation) that are important in Thai. Deeper stacks improve modeling capacity for complex acoustics while residuals maintain trainability.

Temporal pooling. We summarize the sequence with mean pooling across time to obtain a fixed-length representation per window. Mean pooling is simple, stable across different window sizes, and works well when the relevant acoustic evidence is distributed over the window. Attention pooling is a viable alternative but increases parameters and complexity.

Classification head. A lightweight multilayer perceptron maps the pooled representation to class logits: one hidden layer of size 1024 with GELU activation and dropout (0.1), followed by a final linear layer to 5 outputs. This head is intentionally small to reduce overfitting on limited data while providing enough capacity to separate profanity types from non-profanity.

Loss and class imbalance. We train with class-weighted cross-entropy, assigning larger weights to rarer classes (and a smaller weight to the dominant “none” class). This makes the model pay more attention to infrequent profanity categories, improving recall without excessively harming precision.

Why these choices. Convolutions compress and de-

noise raw waveforms efficiently; transformers then model long-range context crucial for speech. Mean pooling and a small classification head keep the model robust and data-efficient. Windowed inference yields timestamps and supports merging/refinement steps (e.g., VAD) that align predictions with censorable spans.

The proposed profanity detection system is based on a transformer architecture, specifically the wav2vec 2.0 model, and operates as follows (see Fig. 2):

This framework leverages the strong representational power of self-supervised transformer models for speech, enabling robust detection of profane content even in low-resource and noisy conditions.

3.5. Evaluation Methodology

Our evaluation framework is designed for real-world censoring use cases and systematic evaluation. It assesses performance across a wide range of temporal parameters and four complementary methodologies, while comparing two stride strategies for sliding-window inference.

Stride strategies. We investigate two ways to advance the analysis window through time:

- **Fixed-time stride:** a constant step of 0.05 s between consecutive windows and strides.
- **Percentage-based stride:** a relative step (e.g., 10%, 20%) of the window length so that overlap scales with window size.

Terminology used in figures. To avoid ambiguity, we use the following shorthand: *eval by 0.05* denotes the fixed-time stride setting with a 0.05 s step, while *eval by percent* denotes percentage-of-window strides (e.g., an 80% stride means a step of $0.8 \times$ window length; for a 0.3 s window, 80% corresponds to a 0.24 s step).

Classification. We evaluate in two label spaces:

- **Binary:** Collapse all profanity classes into a single positive label (“profanity”) versus *none* as negative.
- **Multiclass:** Use the full set of labels {none, yed, guu, meung, hia} and compute F1 in this five-class setting.

3.5.1. Window Evaluation

This is the most direct protocol and operates at the window level. For each audio file, we pre-process the ground truth by labeling an analysis window as *profanity* if its temporal overlap with a profane word span is at least 50%. The model then classifies each window, and predictions are compared to the corresponding ground-truth window labels. Performance is reported with standard classification metrics (e.g., Accuracy, F1-score) at the window level.

3.5.2. Merged-Word Evaluation

This method reconstructs a single profanity event from multiple, consecutive window predictions under the assumption that a spoken profane word may span several overlapping windows. Consecutive windows predicted as the same positive class are merged into a continuous time segment. For example, with a 0.5 s window and 0.25 s stride, four overlapping positive windows covering [2.00, 3.25] s would merge into a single predicted profanity event [2.00, 3.25] s. The merged prediction is then compared against the ground-truth word span to evaluate whether the complete duration was captured.

3.5.3. Merged-Word IoU Evaluation

This variant enhances the merging approach by using Intersection-over-Union (IoU) as the primary criterion for a successful detection. After merging consecutive positive windows into a predicted word instance, we compute IoU with the ground-truth word span; the prediction is counted as a true positive (TP) if IoU exceeds a threshold (e.g., 0.5). Prediction gaps are treated as separators: non-contiguous positives yield distinct predicted word instances, each evaluated independently with its own IoU and TP/FP decision. For instance, consistent predictions that fully tile a ground-truth span yield $\text{IoU} = 1.0$, whereas [positive, none, positive] produces two separate instances with separate IoUs.

3.5.4. Merged-Word IoU Area Combination Evaluation

This region-centric protocol evaluates all predictions associated with a single ground-truth instance collectively. For each ground-truth profanity span, we combine the total duration of all predicted profanity segments that fall within that span, even if they are discontinuous, and compute a single IoU for the region. Example: predictions [2.00, 3.00] and [4.00, 5.00] within a ground-truth span [2.00, 5.00] yield an intersection of 2.00 s and a union of 3.00 s, so $\text{IoU} = 2.00/3.00 \approx 0.67$. With an IoU threshold of 0.5, this is counted as a TP for the entire ground-truth region. This method rewards models that identify substantial portions of a profane word even if predictions are fragmented.

4. Results

We conducted a broad search over window sizes (0.3–2.0 s) and both stride strategies, evaluating four methodologies to understand temporal trade-offs and identify operating points suitable for censoring.

4.1. Model Performance

4.1.1. Evaluation Methodology Selection

Before demonstrating the results, we first selected the most appropriate evaluation methodology. We implemented and compared four approaches: (1) Window Evaluation (direct window classification), (2) Merged-Word Evaluation (merge overlapping positives into word spans), (3) Merged-Word IoU Evaluation (merge then match by IoU threshold), and (4) Merged-Word IoU Area Combination Evaluation (region-centric IoU over all predictions within a ground-truth span). An important observation is that plain Merged-Word Evaluation can artificially inflate F1-scores by focusing evaluation only on regions near annotated profane words, which biases precision upward. To avoid this bias and ensure objective temporal matching, we select the Merged-Word IoU Area Combination protocol as our primary evaluation. It computes IoU correctly at the region level by summing all matching predicted spans within a ground-truth segment (robust to short gaps/fragmentation), while retaining an interpretable overlap measure. In deployment, we still present plain merged-word outputs to users because contiguous, merged spans align best with real-world censoring workflows, whereas the IoU Area Combination score is a fairer, objective measure of detection overlap for model evaluation and windowing selection. Overall, the IoU-based protocols provide objective overlap quantification, reduce evaluation bias, are reproducible, align with temporal detection literature, and work well with VAD for censoring.

4.1.2. Window Size Optimization

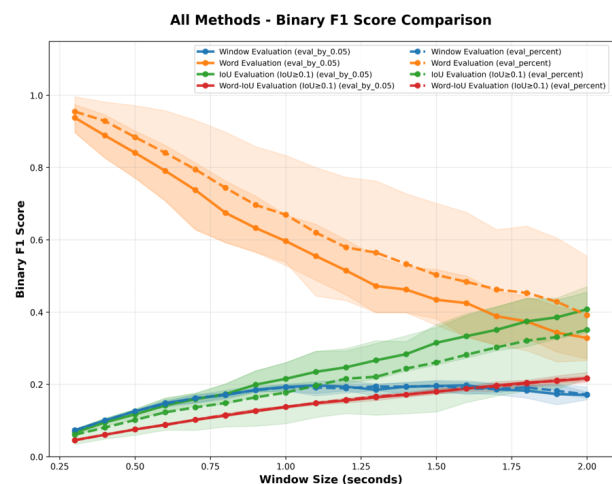


Fig. 3. Binary F1-score eval 0.05 and eval by percent comparison

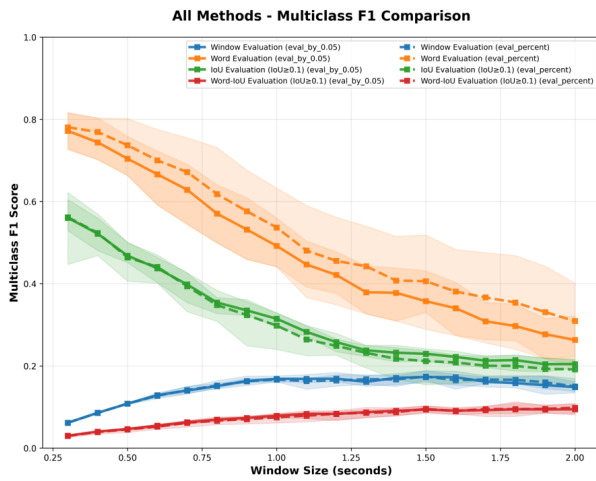


Fig. 4. Multiclass F1-score eval by 0.05 and eval by percent comparison

4.1.3. Stride Selection

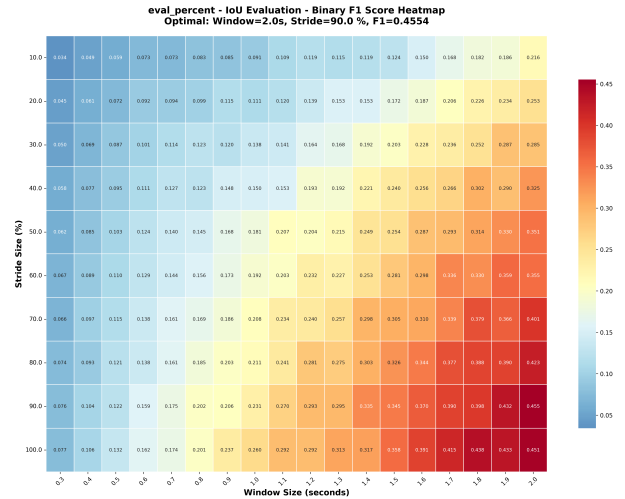


Fig. 5. Binary F1-score eval by percentage

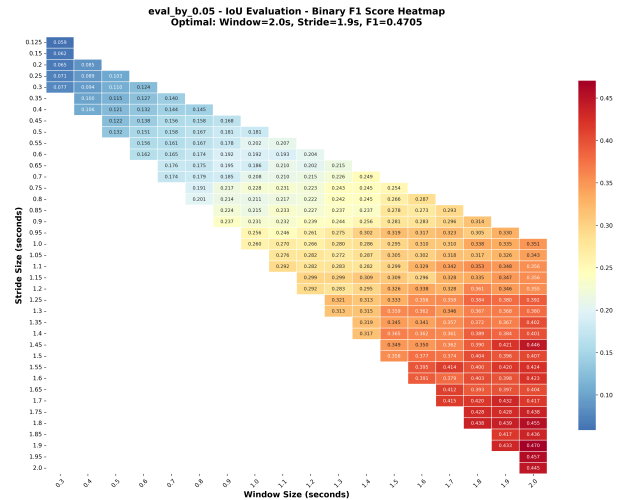


Fig. 6. Binary F1-score eval by 0.05

We analyzed window size effects by plotting F1 versus window length for each evaluation method and stride strategy. Aggregating across stride strategies and methods, we compared F1 versus window size in Figs.3 and 4. Trends include: (i) plain Merged-Word Evaluation tends to report the highest F1 due to the aforementioned bias (orange); (ii) the IoU Area Combination method attains moderate F1 and provides a more faithful temporal measure (green); (iii) Window Evaluation produces consistent but lower F1 since it penalizes boundary fragmentation. In our runs, the IoU Area Combination method shows an operating point near 1.2s that balances binary and multiclass F1; shorter windows favor precise localization, whereas longer windows trade temporal precision for detection robustness(blue); (iv) Merged-Word IoU Evaluation shows the lowest performance with Binary F1 scores around 23% and Multiclass F1 scores approximately 11%. This method appears to be the most restrictive in terms of detection criteria (red).

In conclusion, word evaluation gave the best F1 score since it could come from duplication case we discovered. Therefore, Merged word IoU combined Evaluation has potential to be used as it is fair and logical for evaluation.

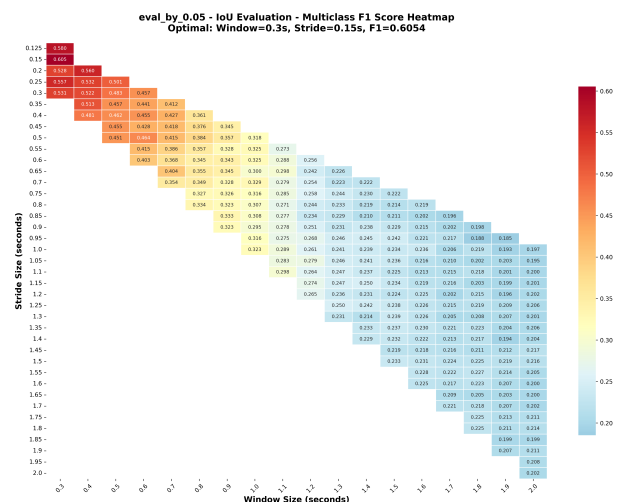


Fig. 7. Multiclass F1-score eval by 0.05

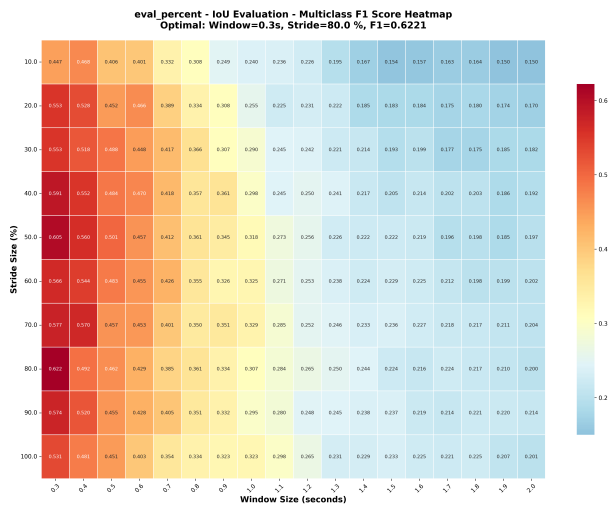


Fig. 8. Multiclass F1-score eval by percentage

We examined stride by heatmaps Figs. 5, 6 over window and stride for both fixed-time and percentage-based advances. For binary classification under the IoU Area Combination protocol, large windows (e.g., 2.0 s) with large strides can yield higher binary F1, reflecting robust presence detection but less precise boundaries. Conversely, multiclass results Figs 7, 8 favor shorter windows with substantial overlap (e.g., 0.3 s with 80% stride), consistent with the need for finer temporal resolution. For the IoU Area Combination method, a 1.2 s window with an $\sim 80\text{--}90\%$ stride yielded a good trade-off in our runs.

4.1.4. IoU Threshold and Percentage Analysis

We performed a threshold sensitivity analysis to select the IoU threshold for the IoU-based methods. As the IoU threshold increases, F1 decreases as expected, since stricter temporal alignment reduces recall. Across binary and multiclass settings, the highest F1 is achieved at a lenient threshold of 0.1, indicating that the model often captures portions of profane spans without perfectly aligning full duration. Therefore, we use $\text{IoU} = 0.1$ for all IoU-based results reported below. Building on this choice, we summarize mean IoU behavior across window/stride configurations as follows.

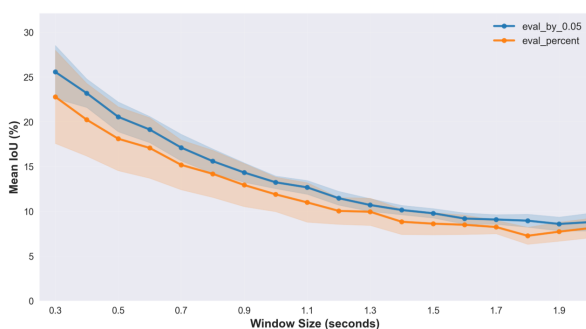


Fig. 9. Mean IoU percentage heatmap across different window and stride configurations

Using the IoU Area Combination metric, we summarized mean IoU across window/stride Figs. 9. Smaller windows (0.3–0.5 s) generally produce higher mean IoU than larger windows, confirming that finer segmentation improves boundary localization. As an illustrative example for a single ground-truth span [2.00, 5.00] s, discontinuous predictions [2.00, 3.00] and [4.00, 5.00] yield an intersection of 2.00 s and a union of 3.00 s, giving $\text{IoU} \approx 0.67$, which would count as a true positive at a 0.1 threshold. Aggregated results show best mean IoU at small windows with appropriately scaled strides, with fixed 0.05 s steps yielding competitive performance at 0.3 s windows.

As illustrated in Fig. 9, there is a clear inverse relationship between window size and mean IoU. Smaller window configurations (0.3–0.5 s) consistently outperform larger windows across both evaluation types. The fixed-step condition (`eval_by_0.05`) shows a slight advantage at small windows, peaking at a combined mean IoU of 29.19% for a 0.3 s window with a 0.25 s stride. Confidence intervals widen for larger windows, indicating higher variance and less consistent temporal localization as window duration increases. These observations align with established time-frequency trade-offs and windowing effects in speech processing [17, 18], and support the hypothesis that shorter analysis windows provide more precise boundary detection for profanity segments.

Focusing on window effects, three operating points highlight the trade-off between detection and temporal localization: a large 2.0 s window with a 1.9 s stride maximizes binary F1 (47.05%) but yields a low IoU (9.16%), indicating coarse boundaries; a short 0.3 s window with an 80% stride gives the best multiclass F1 (52.34%) with improved IoU (24.90%); and a 0.3 s window with a 70% stride attains the highest IoU (25.34%) with strong multiclass F1 (51.17%). Overall, 0.3 s windows consistently tighten temporal boundaries and improve IoU, while very large windows favor presence detection at the expense of precise spans.

4.2. Real-World Usage Recommendations

Recommendations depend on deployment goals:

- Presence detection for censoring decisions (binary): larger windows (e.g., 2.0 s) with IoU-based merging often yield higher binary F1, and VAD post-processing can further tighten spans.
- Timestamp accuracy and localization (multiclass or span extraction): shorter windows (e.g., 0.3 s) with substantial overlap perform better on IoU and multiclass F1; use merged-word IoU methods with a low IoU threshold (0.1) and optional VAD refinement.
- Balanced operating point: under the IoU Area Combination protocol, a 1.2 s window with $\sim 80\text{--}90\%$

stride provided a reasonable balance between binary and multiclass F1 in our experiments.

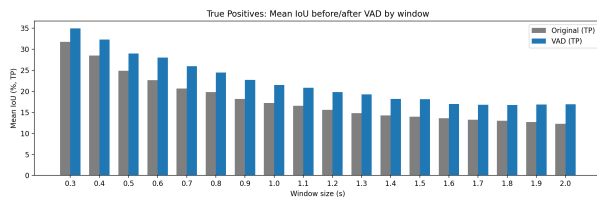


Fig. 10. Voice Activity Detection (VAD) IoU improvement results across different window

As shown in Fig. 10, VAD post-processing consistently improves IoU across all window sizes by trimming silence and bridging brief gaps. The most significant gains occur at moderate window lengths (0.5–2.0 s), where VAD can effectively refine boundaries without excessive fragmentation. For very short windows (0.3 s), VAD still provides modest IoU improvements, while for large windows (2.0 s), the impact is less pronounced due to already coarse boundaries. Overall, VAD is a valuable addition to enhance temporal precision in profanity detection outputs [21].

5. Future Work and Limitations

5.1. Future Work

Context-aware hierarchical models. Our current approach is purely acoustic. A promising direction is a two-stage system in which the acoustic model proposes profanity candidates and a language model performs context-aware verification, helping distinguish genuine slurs from innocuous mentions.

Advanced boundary refinement with neural VAD. We currently use an energy-based VAD for boundary cleanup. A neural VAD trained to separate speech from complex background noise (music, traffic) could improve timestamp precision and reduce false trims.

Real-time streaming architecture. Extending the offline pipeline to streaming requires buffer management, low-latency inference, and on-the-fly windowing/merging, enabling applications like live broadcast moderation.

5.2. Limitations

Sensitivity to noise. Energy-based VAD can be brittle in noisy conditions, and loud non-speech sounds may be misclassified as speech; very quiet profanity may be missed.

Lack of linguistic context. Without text or semantics, the system censors based solely on acoustic cues and cannot disambiguate quoted or educational usage from insults.

Data scarcity. Limited training data constrains generalization, especially across colloquial variants. Enlarging and diversifying the dataset remains a central need.

6. Conclusion

We present a Thai profanity detection system and comprehensive parameter study using a fine-tuned wav2vec 2.0 model. Our framework spans preprocessing, sliding-window inference, merging, IoU-based evaluation, and optional VAD refinement, providing a practical pipeline for automated censoring. Through extensive experiments over windows and strides, we identify evaluation biases in plain merging, motivate IoU-based methodologies with a 0.1 threshold, and offer scenario-specific configurations: large windows for presence detection, short windows for precise localization, and a balanced 1.2 s operating point for mixed objectives. While dataset scale and noise sensitivity limit performance, the system demonstrates the viability of automated profanity moderation in Thai audio and establishes a reproducible foundation for future work in context-aware modeling, neural VAD, and real-time streaming.

References

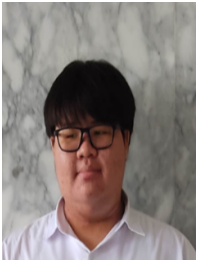
- [1] T. Davidson, D. Warmesley, M. Macy, and I. Weber, “Automated hate speech detection and the problem of offensive language,” *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 11, no. 1, pp. 512–515, May 2017. [Online]. Available: <https://ojs.aaai.org/index.php/ICWSM/article/view/14955>
- [2] A. S. B. Wazir, H. A. Karim, H. S. Lyn, M. F. Ahmad Fauzi, S. Mansor, and M. H. Lye, “Deep learning-based detection of inappropriate speech content for film censorship,” *IEEE Access*, vol. 10, pp. 101 697–101 715, 2022.
- [3] N. Theera-Umporn, S. Chansareewittaya, and S. Auephanwiriyakul, “Phoneme and tonal accent recognition for thai speech,” *Expert Systems with Applications*, vol. 38, no. 10, pp. 13 254–13 259, 2011. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417411006695>
- [4] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” 2020. [Online]. Available: <https://arxiv.org/abs/2006.11477>
- [5] A. Suwanbandit, “Improving thai-dialect automatic speech recognition with curriculum learning,” *Chulalongkorn University Theses and Dissertations (Chula ETD)*, 2024. [Online]. Available: <https://digital.car.chula.ac.th/chulaetd/11546>
- [6] I. López-Espejo, Z.-H. Tan, J. H. L. Hansen, and J. Jensen, “Deep spoken keyword spotting: An overview,” *IEEE Access*, vol. 10, pp. 4169–4199, 2022.

- [7] A. Mesaros, T. Heittola, and T. Virtanen, "Metrics for polyphonic sound event detection," *Applied Sciences*, vol. 6, no. 6, 2016. [Online]. Available: <https://www.mdpi.com/2076-3417/6/6/162>
- [8] S. Hemtanon, K. Phetkrachang, and W. Yangyuen, "Identifying text-based online thai hate speech in social media," *Journal of Information Science Research and Practice*, vol. 42, no. 4, p. 44–56, Sep. 2024. [Online]. Available: <https://so03.tci-thaijo.org/index.php/jiskku/article/view/278073>
- [9] R. Aljumily, "An experiment in keyword-based approach for hate speech detection in on-line social media comments," *Journal of Information Security*, 2023. [Online]. Available: <http://dx.doi.org/10.2139/ssrn.4651992>
- [10] C. Jacobs, N. C. Rakotonirina, E. A. Chimoto, B. A. Bassett, and H. Kamper, "Towards hate speech detection in low-resource languages: Comparing asr to acoustic word embeddings on wolof and swahili," 2023. [Online]. Available: <https://arxiv.org/abs/2306.00410>
- [11] K. J. Piczak, "Environmental sound classification with convolutional neural networks," in *2015 IEEE 25th International Workshop on Machine Learning for Signal Processing (MLSP)*, 2015, pp. 1–6.
- [12] A. Conneau, A. Baevski, R. Collobert, A. Mohamed, and M. Auli, "Unsupervised cross-lingual representation learning for speech recognition," 2021. [Online]. Available: <https://arxiv.org/abs/2006.13979>
- [13] A. Babu, C. Wang, A. Tjandra, A. Astafurov, W.-N. Hsu, A. Lee, A. Ali, and A. Conneau, "Xls-r: Self-supervised cross-lingual speech representation learning at scale," 2021. [Online]. Available: <https://arxiv.org/abs/2111.09296>
- [14] T. N. Sainath and C. Parada, "Convolutional neural networks for small-footprint keyword spotting," in *Interspeech 2015*, 2015, pp. 1478–1482.
- [15] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018. [Online]. Available: <https://arxiv.org/abs/1804.03209>
- [16] J. An, W. Lee, Y. Jeon, J. Ok, Y. Kim, and G. G. Lee, "An investigation into explainable audio hate speech detection," in *Proceedings of the 25th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, T. Kawahara, V. Demberg, S. Ultes, K. Inoue, S. Mehri, D. Howcroft, and K. Komatani, Eds. Kyoto, Japan: Association for Computational Linguistics, Sep. 2024, pp. 533–543. [Online]. Available: <https://aclanthology.org/2024.sigdial-1.45/>
- [17] F. J. Harris, "On the use of windows for harmonic analysis with the discrete fourier transform," *Proceedings of the IEEE*, vol. 66, no. 1, pp. 51–83, 1978.
- [18] J. B. Allen, "Short-time spectral analysis, synthesis, and modification by discrete fourier transform," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 25, no. 3, pp. 235–238, 1977.
- [19] U. Shrawankar and V. Thakare, "An adaptive methodology for ubiquitous asr system," 2013. [Online]. Available: <https://arxiv.org/abs/1303.3948>
- [20] B. McFee, C. Raffel, D. Liang, D. P. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in python," in *Proceedings of the 14th Python in Science Conference*. SciPy, 2015, pp. 18–25.
- [21] J. Sohn, N. S. Kim, and W. Sung, "A statistical model-based voice activity detection," *IEEE Signal Processing Letters*, vol. 6, no. 1, pp. 1–3, 1999.
- [22] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [23] T. Ko, V. Peddinti, D. Povey, and S. Khudanpur, "Audio augmentation for speech recognition," in *Proc. Interspeech*, 2015, pp. 3586–3589.
- [24] D. S. Park, W. Chan, Y. Zhang, C.-W. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019, pp. 2613–2617.





Nutchapol Rodpholchoo was born in Bangkok, Thailand, in 2003. He is currently pursuing a B.Sc. degree in Computer Science at Assumption University, Samut Prakan, Thailand, where his major field of study is computer science. His academic interests include artificial intelligence, programming languages, cloud computing, and computer architecture. In addition, he served as a committee member of the Faculty of Science and Technology at Assumption University, where he contributes to promoting academic activities and enhancing student engagement in information technology and innovation and he was batch leader of 2022 Students which focused on public relation with the same years students.



Nutchanon Tongkumbanchong was born in Bangkok, Thailand, in 2003. He is currently pursuing a B.Sc. degree in Computer Science at Assumption University, Samut Prakan, Thailand, where his major field of study is computer science. His academic interests include artificial intelligence, programming languages, cloud computing, and computer architecture. In addition, he served as a committee member of the Faculty of Science and Technology at Assumption University, where he contributes to promoting academic activities and enhancing student engagement in information technology and innovation. He is an active student member of Assumption University's academic community and has participated in several research and technology-oriented projects.



Sirinalin Saekhow was born in Bangkok, Thailand in 2004. She is currently pursuing a B.Sc. degree in Computer Science at Assumption University, Samut Prakan, Thailand, where her major field of study is computer science. Her academic interests include artificial intelligence, computer vision, natural language processing, and computer network. Ms. Sirinalin Saekhow is an active student member of Assumption University's academic community and has participated in several research and technology-oriented projects.